```
#Step 1 Import necessary libraries
import tensorflow as tf
from tensorflow.keras import layers, models

#Step 2 Load and preprocess MNIST dataset
(X_train, y_train), (X_test, y_test) = tf.keras.datasets.mnist.load_data()
X_train, X_test = X_train / 255.0, X_test / 255.0
X_train = X_train.reshape(-1, 28, 28, 1)
X_test = X_test.reshape(-1, 28, 28, 1)
y_train = tf.keras.utils.to_categorical(y_train, 10)
y_test = tf.keras.utils.to_categorical(y_test, 10)

#Step 3 Define the CNN model architecture
model = models.Sequential()

#Step 4
# Phase 1: Convolution + Activation (ReLU)
model.add(layers.Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)))

# Phase 2: Pooling
model.add(layers.MaxPooling2D((2, 2)))

# Phase 3: Convolution + Activation (ReLU)
model.add(layers.Conv2D(64, (3, 3), activation='relu'))

# Phase 4: Pooling
model.add(layers.MaxPooling2D((2, 2)))

# Phase 5: Convolution + Activation (ReLU)
model.add(layers.Conv2D(128, (3, 3), activation='relu'))

# Phase 6: Pooling
model.add(layers.MaxPooling2D((2, 2)))

#Step 5 Flatten the output from the convolutional layers
model.add(layers.Flatten())

#Step 6 Fully Connected Layer (Dense)
model.add(layers.Dense(128, activation='relu'))

#Step 7 Output Layer (for multi-class classification)
model.add(layers.Dense(10, activation='softmax'))  # 10 classes for MNIST

#Step 8 Compile the model
model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])

#Step 9 Display model summary
model.summary()

#Step 10 Train the model
model.fit(X_train, y_train, epochs=10, batch_size=32, validation_data=(X_test, y
```

```
model.fit(X_train, y_train, epochs=10, batch_size=32, validation_data=(X_test, y

#Step 11 Evaluate the model
test_loss, test_acc = model.evaluate(X_test, y_test, verbose=2)
print(f'Test accuracy: {test_acc}')
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-da
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
/usr/local/lib/python3.11/dist-packages/keras/src/layers/convolutional/base
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential"
```

Layer (type)	Output Shape	
conv2d (Conv2D)	(None, 26, 26, 32)	
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	
conv2d_1 (Conv2D)	(None, 11, 11, 64)	
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	
conv2d_2 (Conv2D)	(None, 3, 3, 128)	
max_pooling2d_2 (MaxPooling2D)	(None, 1, 1, 128)	
flatten (Flatten)	(None, 128)	
dense (Dense)	(None, 128)	
dense_1 (Dense)	(None, 10)	

```
 Total params: 110,474 (431.54 KB)
 Trainable params: 110,474 (431.54 KB)
 Non-trainable params: 0 (0.00 B)
Epoch 1/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 68s 35ms/step - accuracy: 0.8709 - loss: 0.4
Epoch 2/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 64s 34ms/step - accuracy: 0.9783 - loss: 0.0
Epoch 3/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 81s 34ms/step - accuracy: 0.9856 - loss: 0.0
Epoch 4/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 86s 36ms/step - accuracy: 0.9890 - loss: 0.0
Epoch 5/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 80s 35ms/step - accuracy: 0.9914 - loss: 0.0
Epoch 6/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 81s 34ms/step - accuracy: 0.9936 - loss: 0.0
Epoch 7/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 84s 36ms/step - accuracy: 0.9939 - loss: 0.0
Epoch 8/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 77s 33ms/step - accuracy: 0.9949 - loss: 0.0
Epoch 9/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 82s 33ms/step - accuracy: 0.9953 - loss: 0.0
Epoch 10/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 85s 35ms/step - accuracy: 0.9955 - loss: 0.0
313/313 - 3s - 11ms/step - accuracy: 0.9868 - loss: 0.0578
Test accuracy: 0.9868000149726868
```